# The Simplified Models of Notebook 07 vs. the Full Library Model

**A reference for understanding (and documenting) what `07_hjb_insurance_optimization.ipynb` abstracts away — and why.**

> Scope & method. This compares the two reduced-form models *inside* notebook 07 against the library's full GAAP modeler. It is based on a direct reading of the committed notebook (cells 5/8/10/16/20/37) and the library source (`manufacturer.py` + its five mixins, `hjb_solver.py`, `hjb_quadrature.py`, `insurance_program.py`, `insurance_pricing.py`, `loss_distributions.py`, `claim_development.py`, `tax_handler.py`, `config/manufacturer.py`). Citations are given as `file.py:line` (library) or `cell N` (notebook). Numbers are from the validated committed run. Where a figure was re-derived from live code it is flagged.

---

## 1. Summary

Notebook 07 does **not** contain one simplified model — it contains **two**, and they play different roles. Both are reduced-form stand-ins for the library's full financial modeler, `WidgetManufacturer`.

| | **Full model** | **Simplified model A** | **Simplified model B** |
|---|---|---|---|
| Name | `WidgetManufacturer` (GAAP) | Simple-cash CRN simulator | HJB PIDE formulation |
| Where | library (`manufacturer.py` + 5 mixins) | notebook cell 10 (`simulate_with_crn`, `step_year_per_event_scheduled`) | notebook cell 20 (`company_dynamics`, `jump_term`, solved by `HJBSolver`) |
| Role | **authoritative / conservative** validation (Parts 10b/10c/10d, Part 5.5 full arm) | **fast experiment engine** (Parts 5, 8, 10, 11, 13; the regret/attribution/sweeps) | **policy generator** — derives the state-dependent retention rule `SIR(A, e)` that the other two deploy |
| State | event-sourced double-entry ledger (~12 accounts) | 2 states: total assets `A`, deferred claim liabilities `Lb` | 2 states: total assets `A`, equity ratio `e = equity/A` |
| Time | annual or monthly, intra-period timing | annual lock-step | continuous-time (compound-Poisson + diffusion), backward-marched at `dt = 0.0125` |
| Ruin | two-tier ASC 205-40 going-concern + liquidity | single equity floor ($100K) + per-event single-loss test | equity floor only (jump "ruin atom"), plus an in-solve N-event admissibility bound |

**The relationship in one sentence.** The HJB (model B) is the *continuous-time analogue* of the simple-cash MC (model A), deliberately calibrated to ask "the same after-tax, equity-based question"; model A is in turn a *reduced-form analogue* of the full GAAP model, deliberately calibrated to match its no-loss growth, leverage, claim timing, and loss exposure. The HJB **derives** the policy, the simple MC **explores/attributes/sweeps** it cheaply, and the full GAAP model is the **conservative check** that the resulting verdict survives real accounting frictions.

**The headline answer on fidelity.** After a sequence of calibrations (issues #1588 base-liability, #1597 claim schedule, #1598 single-loss liability, #1607 size-scaled severity, #1625 revenue-scaled facility, #1637 LOC collateral, #1648 leverage amplification), the three models now agree on **direction** (ruin falls with coverage; insurance is value-creating in this regime) and broadly on **no-loss growth** (Part 10c). They still differ in **magnitude**: the simple model shows a large, clean insurance advantage ( `+2.98 pp/yr` g_CE at $5M, 100% of walk-forward windows), while the full GAAP model shows a real but *thin* advantage ( `+0.79 ± 0.26 pp/yr`, ~3 SE). That residual gap is exactly the set of frictions model A omits (liquidity channel, gross-loss-against-equity timing, richer insolvency) — see §7.

**On your specific question — yes, the HJB jump term is a simplification**, and a multi-layered one (continuous-time compound-Poisson; equity-only block-diagonal operator; severity discretized to quadrature atoms; an analytical retained-loss formula that collapses the 4-layer tower; a single mean-matched exponential claim-paydown in place of the 10-year schedule; and an explicit IMEX numerical treatment). It is dissected term-by-term in §5.1 and Appendix D.

---

## 2. The three models at a glance

| Dimension | Full `WidgetManufacturer` | Simple-cash MC (cell 10) | HJB PIDE (cell 20) |
|---|---|---|---|
| Balance sheet | cash, AR, inventory, prepaid, insurance receivables, gross/net PP&E, DTA/DTL, AP, accrued, short-term borrowings, claim liabilities | `A` (assets), `Lb` (claim liab.); `equity = A·(1−k) − Lb`, `k = 0.222` | `A`, `e`; `equity = e·A`; claim liab. implied `Lb = A·max(1−k−e, 0)` |
| Revenue | `(total_assets − net_DTA − insurance_recv) · ATR · shock` | `A · ATR · shock` | drift uses `A · MU`, `MU = ATR·margin` |
| Retention/tax | true ASC 740 NOL (80% cap), DTA valuation allowance, DTL, quarterly timing | flat `PHI = retention·(1−tax) = 0.525`, applied to income **and** losses | same flat `PHI = 0.525` |
| Loss model | `ManufacturingLossGenerator` (per claim) | **same** generator, CRN pool, thinned | **same** severities → quadrature atoms |

| Dimension | Full `WidgetManufacturer` | Simple-cash MC (cell 10) | HJB PIDE (cell 20) |
|---|---|---|---|
| Tower & pricing | notebook `make_program` + Wang `LayerPricer` (sweep arm) | notebook `make_program` + Wang; routed through `InsuranceProgram.process_claim()` | analytical `L_ret = (1+LAE)(min(X,SIR)+(X−tower)+)`; premium from a precomputed 2-D table |
| Claim payout | `ClaimDevelopment.create_long_tail_10yr` | **same** 10-yr schedule (explicit buffer) | mean-matched single exponential `β ≈ 0.26–0.28/yr` |
| Liquidity/cash | full: facility draw, LOC fees, intra-year cash trough | **none** (no cash account) | **none** (no cash dimension) |
| Insolvency | 2-tier ASC 205-40 + intra-period + compulsory-premium | equity < $100K, or single loss > equity | post-jump equity < $100K (ruin atom); N-event bound in-solve |
| Output | full financials, `is_ruined` | equity paths → ruin %, `g_CE` | growth function `V(A,e)` + optimal `SIR(A,e)` |
| Speed | slow (Decimal ledger, ~43 s for 1000 paths) | fast (vectorized cash recursion) | one solve (~minutes, Colab) → reused everywhere |

---

## 3. Commonalities (what is shared / deliberately kept identical)

The notebook works hard to keep the two simplified models **calibrated to** the full model so comparisons are apples-to-apples. The shared elements:

1. **The loss model is literally the same object.** All three consume `ManufacturingLossGenerator` with the identical `LOSS_PARAMS` (cell 10): attritional `freq 1.25`, Lognormal(mean $10K, cv 5); large `freq 0.4`, Lognormal(mean $450K, cv 2.5); catastrophic `freq 0.09`, Pareto(α 2.1, xm $800K). The simple MC draws a CRN pool and thins it; the HJB turns the same severities into quadrature atoms ( `component_atoms` , `hjb_quadrature.py:177` ); the full-model sweep arm regenerates the same pool with the same seeds. `λ = Σ freq ≈ 1.74` events/yr is shared.
2. **The insurance tower is the same 4-layer structure**, built by the notebook's `make_program` (cell 10): SIR → $5M primary, 5*M*→25M, 25*M*→50M, 50*M*→200M cat ( `TOWER_TOP = 200M` ).

The simple MC **and** the full-model sweep both route every loss through the *library's* `InsuranceProgram.process_claim()` ( `insurance_program.py:718` ), so the retained-vs-recovered split is computed by the real tower (retained = below-SIR + above-tower). (The HJB approximates this — see §5.1d.)

3. **Premiums use the same Wang-transform distortion pricer** (cell 10, `make_layer_pricers` + `_wang_layer_loss` ): `g(u) = Φ(Φ⁻¹(u) + λ)` , with a single `λ` back-solved so the primary working layer prices at `TARGET_LOSS_RATIO = 0.65` . *Re-derived live:* `WANG_LAMBDA ≈ 0.3669` , giving monotone tower loadings **1.54× / 2.84× / 3.80× / 4.43×** (implied LR 65% / 35% / 26% / 23%). The HJB drift reads premiums from a precomputed `premium(SIR, A)` table built with the *same* pricer (cell 20:202).
    - Note: this Wang pricer is **notebook-local** (cell 10), *not* the installable library pricer. The library's `InsurancePricer` ( `insurance_pricing.py` ) is a different, simulation+ `Premium=Pure/(1−V−Q)` actuarial-identity pricer with ALAE/ULAE loadings. The notebook deliberately uses the analytical Wang distortion instead (see §6.6).
4. **After-tax retention is identical.** Simple MC and HJB both apply `PHI_RETENTION = retention·(1−tax) = 0.70·0.75 = 0.525` to operating income net of premium, *and* to retained losses (loss tax-shield). The full model derives the same effect endogenously (70% retention / 30% dividend; ASC 740 tax).
5. **The base-liability / leverage calibration is shared.** Both simplified models carry a permanent non-claim base liability `B = k·A` , `k = BASE_LIAB_RATIO = 0.222` , so `equity = A·(1−k) − Lb` . This is *calibrated* to the full model's no-loss steady-state equity ratio `e ≈ 0.778` (which the full model reaches via working-capital facility + DTL). The full model is seeded to start there too ( `initial_base_liability_ratio = 0.222` , #1645). The matching `1/(1−k)` leverage amplification (#1648) makes no-loss equity in all three compound at `PHI·MU/(1−k) ≈ 12.6%/yr` .
6. **The claim-development schedule matches exactly (simple MC ↔ full).** The simple MC books each retained loss on the explicit 10-year schedule `[0.10, 0.20, 0.20, 0.15, 0.10, 0.08, 0.07, 0.05, 0.03, 0.02]` via a rolling buffer ( `step_year_per_event_scheduled` , cell 10), which **is**

`ClaimDevelopment.create_long_tail_10yr` (`claim_development.py:93`; mean lag ≈ 3.84 yr). (The HJB approximates this with one exponential — §5.2.)

7. **Size-scaled severity (#1607) is applied consistently** at `GAMMA_SEV = 0.5`: large/cat per-event severity scales `(A / $5M)^γ` with exposure-preserving `(1−γ)` frequency. Simple MC (`size_severity_factor`, cell 10), HJB (`build_equity_jump_operator_2d_sizescaled`, `hjb_quadrature.py:352`), and the full-model sweep all use it. (The library supports severity scaling natively, `loss_distributions.py:22`, default off.)
8. **LAE is `0.12` everywhere** — a flat markup on retained loss in the simple MC and HJB; the full model's `lae_ratio = 0.12` per ASC 944-40 (`manufacturer_claims.py`).
9. **The ergodic objective and ruin floor are shared.** The decision metric `g_CE` (certainty-equivalent log-growth — full explainer in §9) floors terminal equity at `RUIN_THRESHOLD = $100K` and takes the geometric mean of log-equity. The HJB's terminal condition is `V(A, e, T) = log(max(e·A, E_FLOOR))` with the **same** `E_FLOOR = $100K` (cell 20:316), and its ruin atom carries `V_RUIN = log(E_FLOOR)`. So the growth function the solver maximizes and the metric the MC reports are the *same* objective, by construction. (`LogUtility`, `hjb_solver.py:252`, is documented as the logarithmic *ergodicity transformation*, not a risk-preference.)
10. **CRN op-income shocks are shared.** All three apply the same lognormal multiplier `exp(σz − σ²/2)`, `σ = REL_OP_INC_VOL = 0.30`; the full-model sweep injects the identical `z` draws via a custom `_SweepCRNShock` process (cell 16).

---

## 4. Simplifications — the simple-cash CRN simulator (model A) vs. the full model

The simple MC compresses the entire GAAP balance sheet into a 2-state cash recursion. The simplifications, in rough order of behavioral impact:

1. **Two states instead of a double-entry ledger.** `equity = A·(1−k) − Lb` is a single algebraic residual; the full model derives `equity = total_assets − total_liabilities` from ~12 ledger accounts (`manufacturer_balance_sheet.py:327`).

Cash, AR, inventory, PP&E, AP, prepaid insurance, insurance receivables, and short-term borrowings simply do not exist as separate states.

2. **No cash / liquidity channel — the single biggest gap (the "#1604" gap).** The simple model has no cash account, no working-capital facility, and therefore **no liquidity-driven ruin**. A firm can only die by equity erosion or a single loss exceeding equity. The full model can ruin a *solvent-on-equity* firm through liquidity: a working-capital-facility breach ( `cash < −facility` , `manufacturer_solvency.py:402` ), an intra-year cash trough ( `estimate_minimum_cash_point` , `:473` ), or inability to pay a compulsory premium ( `manufacturer_claims.py:77` ). Because premium is a recurring cash outflow, this channel historically made insurance look *worse* in the full model until #1625/#1637 fixed it (§7).
3. **Insolvency is a single equity floor, not ASC 205-40.** Simple: ruin iff year-end `equity < $100K` , or a single retained loss (gross of LAE) exceeds current equity ( `step_year_per_event_scheduled` , #1598). Full: a two-tier test — hard stops (equity ≤ 0; facility breach) **plus** a multi-factor going-concern assessment requiring ≥2 of {current ratio < 1.0, DSCR < 1.0, equity ratio < 5%, cash runway < 3 months} ( `manufacturer_solvency.py:361` ), plus intra-period and compulsory-premium checks.
4. **Tax is a flat multiplier, not ASC 740.** Simple/HJB apply `PHI = 0.525` symmetrically and immediately, including a full, immediate **loss tax-shield** ( `Lb += PHI·r_gross` ). The full model runs real NOL carryforwards with the 80% TCJA limitation, a DTA with a graduated valuation allowance (50%/75%/100% after 3/4/5 consecutive loss years), DTLs from accelerated tax depreciation, and quarterly accrual/payment timing ( `tax_handler.py` ). So the simple model's loss shield is *immediate and unconditional*, where the full model's is *deferred, capped, and can be written off*.
5. **The base liability is a flat proportion that never pays down.** `B = k·A` scales with the firm and is permanent (cell 10). In the full model the corresponding real liabilities (a drawn revolver + DTL) are dynamic — they grow, shrink, and can breach the facility.
6. **No PP&E, depreciation, capex, working-capital cycle, or revenue-recognition timing.** The full model runs straight-line depreciation (10-yr life), maintenance capex ( `capex = depreciation` ), DSO/DIO/DPO working-capital day-ratios

(45/60/30), and ASC 606 revenue recognition. The simple model's revenue is just `A·ATR·shock`; the full model's revenue base *excludes* net DTA and insurance receivables (`manufacturer_income.py:52`).

7. **Insurance recovery is immediate; no receivables.** Simple MC nets recovery inside `process_claim`. The full model books an insurance *receivable* (ASC 310/410-30) and collects the cash later — a timing drag absent from the simple model.
8. **Annual lock-step, no sub-annual timing.** No premium-payment month, no quarterly tax outflows, no within-year revenue pattern — all of which the full model uses to find the cash trough that the facility must bridge.
9. **Leverage amplification is hard-coded, not emergent.** The simple model multiplies retained earnings by `1/(1−k)` (cell 10, #1648) so equity compounds at the full model's empirical rate; the full model produces that leverage naturally through its balance sheet.
10. **No letter-of-credit mechanics.** The full model's default `sir_collateral_mode = "letter_of_credit"` charges an LOC fee on outstanding reserves and pays the retention from operating cash over the schedule; the simple model just defers the retained loss as `Lb` with no carry fee.

---

## 5. Simplifications — the HJB formulation (model B)

The HJB is a *further* simplification: it compresses even the simple MC's annual recursion into a continuous-time PIDE over `(A, e)` so that dynamic programming becomes tractable. Its simplifications fall into the loss/jump structure, the diffusion, the state space, and the numerics.

### 5.1 The jump term (your example) — dissected

The loss term is `λ · E_X[V(post-jump) − V]` (`make_jump_term_2d`, `hjb_quadrature.py:493`). It is a simplification in **six** distinct ways:

- **(a) Continuous-time compound Poisson, not a discrete annual aggregate.** Losses arrive as a memoryless Poisson process at rate `λ(A)`; the simple MC and full model process a *discrete set of events per year*. The PIDE convention (see `feedback_hjb_pide_convention.md`) requires the jump to be

**uncompensated** — `λ·E[V(w−L)−V(w)]` carries the full linear loss drag — and the drift must *not* subtract `λ·E[L]` (doing so once collapsed `SIR*` to the grid floor).

- **(b) Equity-only, block-diagonal-in-assets operator.** A loss reduces equity only — `e → e − ϕ·L_ret/A` — with the **assets index unchanged at impact** (a loss books a claim liability; assets stay ~flat). So the operator is one independent 1-D-in-`e` map per assets-slice, assembled as `block_diag(...)` (`hjb_quadrature.py:271`). This keeps it ~1 GB instead of the ~100s of GB a full bilinear (assets-moving) jump would need. The asset drawdown of paying the loss is handled *separately and smoothly* by the drift's `−β·Lb` term, not by the jump.
- **(c) Severity discretized into quadrature atoms.** `E_X[·]` is computed over a fixed atom table: 32-node Gauss-Hermite for the lognormal components (exact for smooth integrands), 300 stratified atoms for the Pareto cat layer (`component_atoms`, `hjb_quadrature.py:177`). Atom-mean error < 0.1%, but it is still a discretization of a continuous severity.
- **(d) An analytical retained-loss formula that collapses the 4-layer tower.** The HJB uses `L_ret = (1+LAE)·(min(X, SIR) + (X − TOWER_TOP)+)` (`hjb_quadrature.py:306`) — retain everything below the SIR plus anything above the tower top. This ignores the tower's **reinstatements, aggregate limits, and exact layer allocation** that the simple MC and full model get from the real `InsuranceProgram.process_claim()`. For a single occurrence with the notebook's per-occurrence layers the two largely coincide, but the HJB cannot see aggregate-limit exhaustion within a year.
- **(e) IMEX / explicit treatment.** The jump is evaluated at `V_old` and folded into the RHS (`hjb_solver.py:1616`), keeping the local operator tridiagonal; it is never part of an implicit solve. Stability needs `λ·dt < 1` (satisfied: `λ·dt ≈ 1.74·0.0125 ≈ 0.02`).
- **(f) Nearest-grid snapping of the control.** The default jump callback snaps `(A, e, SIR)` to the nearest grid nodes (`interp="nearest"`, `hjb_quadrature.py:632`). This is *exact* during the solve (which only ever evaluates on nodes) but is a discretization for off-grid deployment.

### 5.2 Single exponential claim-paydown instead of the 10-year schedule

The HJB drift pays claim liabilities down at a single mean-matched exponential rate `β = BETA_PAYDOWN ≈ 0.26–0.28/yr` ( `A_dot = … − β·Lb` ), because tracking claim *age* would add an intractable third state dimension. The simple MC and full model use the actual 10-year `CLAIM_DEV_FACTORS` schedule. This HJB↔MC timing mismatch is deliberate; the post-Part-11 diagnostic (cell 56) confirms the `g_CE` -optimal SIR barely moves between the two paydown shapes.

### 5.3 Diagonal diffusion (dropped A↔e correlation)

The operating-income shock perfectly correlates the assets and equity-ratio channels, but the solver supports no cross-diffusion term, so `company_diffusion` returns only the diagonal `[σ_A², σ_e²]` (cell 20:261). This is a second-order approximation, justified because the loss **jump** — not the smooth diffusion — dominates the risk.

### 5.4 The 2-D state itself

Collapsing the whole balance sheet to `(A, e)` is the core modeling bet. It is richer than a 1-D "wealth" HJB (which recommended retentions larger than equity, because it could not see the assets/equity gap), but it still cannot represent cash/liquidity. Consequently the HJB has **no liquidity-ruin channel** — ruin is purely the equity floor via the jump's ruin atom. This is the deepest HJB simplification and the reason Part 10b (full model) remains the conservative test.

### 5.5 The N-event admissibility bound — a simplification *and* a fix

Because the HJB's smooth, continuous-wealth objective cannot see (i) the discrete GAAP equity-insolvency cliff (a single loss can end the firm mid-year) or (ii) cumulative multi-loss ruin over 25 years, the raw solver would recommend self-insuring ~0.5–0.65 of *assets* — survivable per single event yet value-destroying. The notebook adds an **in-solve hard constraint** ( `control_feasibility = sir_multi_loss_feasible` , cell 20:332): `(1+LAE)·N·SIR ≤ E − E_FLOOR` , i.e. `SIR ≤ T/N` with `N = λ ≈ 1.74` , landing at `T/N ≈ 0.51·equity` ( `multi_loss_insolvency_retention_cap` , `hjb_quadrature.py:770` ). This is knob-free (derived from the loss frequency) and compensates for what the continuous objective omits — see §6.2.

### 5.6 Explicit scheme at fixed `dt` (no 2-D implicit)

The solver has no 2-D implicit/Crank-Nicolson path (`can_use_implicit = use_implicit and ndim == 1`, `hjb_solver.py:1652`), so the 2-D solve uses explicit Euler at `dt = 0.0125` with `auto_cfl = False` (the global auto-CFL trial is pathologically conservative on a log-assets grid; #1611). A NaN/Inf guard and a realized-local-CFL diagnostic backstop stability. This is a numerical simplification relative to an unconditionally-stable implicit solve.

### 5.7 Premium by interpolated table

The drift reads `premium(SIR, A)` from a precomputed grid interpolated in `(log A, log SIR)` (cell 20:208), rather than repricing each evaluation — a table approximation (exact on grid nodes).

### 5.8 No ASC-740 tax detail

Like the simple MC, the HJB uses the flat `PHI = 0.525`; no NOL/DTA/DTL.

---

## 6. Enhancements in the simplified models (beyond runtime)

The simplified models are not merely faster — several add capabilities the full model does not have:

1. **Optimization itself (the biggest one).** The full `WidgetManufacturer` is a *simulator*: it runs a given strategy. The HJB **derives** a globally-optimal, **state-dependent feedback policy** `SIR(A, e)` via dynamic programming (`solve_finite_horizon`, `hjb_solver.py:1271`). Nothing in the library full model produces an optimal control; the HJB is a genuine addition, not a reduced full model.
2. **A knob-free, principled retention bound.** The N-event survivability bound `T/N ≈ 0.51·equity` (#1659) is *derived from the modeled loss frequency*, retiring the earlier hand-set `κ = 0.50` economic cap. The full model imposes no such admissibility structure — you may set any SIR.
3. **A decision-grade single-axis metric.** `g_CE` (certainty-equivalent log-growth, ruin-floored at $100K) collapses survivor-growth and ruin into one comparable number — the right object for a single-decision choice (averaging only

survivors is the wrong reference class). The full model emits raw financials; the notebook's metric layer is an analytical enhancement.

4. **CRN paired variance reduction.** "Same storms, different ships": every strategy faces identical loss draws and op-income shocks, so differences are causal and detectable with ~10× fewer paths. This is an experimental-design enhancement layered on top of (and exploited by) the simple MC.
5. **Exact attribution counterfactual.** The simple MC exposes a `free_premium` switch (cell 10), enabling an *exact* CRN-paired decomposition of the g_CE advantage into ruin-avoidance + volatility-trimming − premium-drag (Part 11b). This kind of controlled counterfactual is impractical in the full model.
6. **Analytical Wang distortion pricing.** The notebook's per-layer Wang loading ( `g(u)=Φ(Φ⁻¹(u)+λ)` , single anchor, size-dependent, `@lru_cache` d) is a more principled tail-pricing scheme for this purpose than the library `InsurancePricer` 's simulation+SD approach (which could run away in the deep tail and needed an ad-hoc loss-ratio floor; #1587→#1601). It is also fully analytical (no Monte Carlo to price a layer).

---

## 7. Behavioral consequences & reconciliation (structural → empirical)

How the §4–§5 simplifications actually move the numbers, and how the notebook reconciles them:

- **No-loss reconciliation (Part 10c, #1642/#1648).** With losses suppressed, the simple model and the full model should grow at the same rate. The `1/(1−k)` leverage amplification was added precisely so the simple no-loss equity drift `PHI·MU/(1−k) ≈ 12.6%/yr` matches the full GAAP modeler (which additionally earns deferred-tax float and depreciation shields). Part 10c prints both with a standard error and flags agreement within Monte-Carlo noise. A no-loss gap would otherwise be misread as an insurance effect.
- **The liquidity channel was the dominant historical artifact.** Before #1625/#1637, the full model used a *fixed* $2M working-capital facility. As the firm grew, that fixed limit shrank relative to revenue, so a larger recurring premium triggered *more* spurious facility-breach ruins — producing a backwards "more coverage → more ruin" ordering that the simple model (with no liquidity channel) never showed. Two

fixes closed it: **#1625** (revenue-scaled facility, 20% of revenue, grows with the firm) and **#1637** (letter-of-credit collateral instead of a 100% upfront cash lock, so a retained loss no longer strictly worsens near-term liquidity). With both, the full model now ruins *less* with more coverage — agreeing with the simple model's direction.

- **The residual magnitude gap (simple ≫ full).** Two structural differences keep the full-model advantage thinner: (i) the full GAAP model charges the **gross** loss against equity in the year it lands (harsher than the simple model's after-tax, deferred treatment), making retention look relatively cheaper; (ii) the liquidity/premium-drain channel still costs insured strategies something the simple model never sees. Net: simple `+2.98 pp/yr` (100% of windows) vs full `+0.79 ± 0.26 pp/yr` (~3 SE, 3/5 windows). The notebook treats the 1000-path full-model Part 10b/10d as authoritative and states the verdict against its standard error.
- **The HJB policy transfers cleanly despite its simplifications.** Because the HJB learns the *growth-vs-ruin trade-off* (not absolute magnitudes), and that trade is robust to the accounting layer, the same `SIR(A, e)` surface is deployed in the simple MC (cell 37) and the full model (Part 10b) and is value-creating in both. The N-event bound (§5.5) is what makes the *raw* policy value-creating rather than self-insuring into the multi-loss-ruin regime.
- **The paydown-shape simplification is empirically negligible.** The HJB's single exponential β vs the MC's 10-year schedule moves the g_CE-optimal SIR by a trivial amount (cell 56 diagnostic) — validating §5.2.

---

## 8. Which model to trust when (practical guidance)

| Question | Use | Why |
|---|---|---|
| "What retention should the firm hold, as a function of its state?" | **HJB** (`SIR(A,e)`) | Only model that *optimizes*; produces the dynamic feedback rule. Trust the *shape/policy*, not its absolute ruin numbers. |
| "How do strategies compare? Where's the regret optimum? Sensitivity? Attribution?" | **Simple-cash MC** | Fast, CRN-paired, controllable counterfactuals. Trust *relative* rankings and the regret band. Optimistic on omitted frictions. |

| Question | Use | Why |
|---|---|---|
| "Is the verdict real once accounting frictions bite? Absolute ruin? Final sign?" | **Full GAAP (Part 10b/10c/10d)** | Conservative, realistic liquidity/tax/insolvency. The authoritative read; state results against the standard error. |
| "Does no-loss growth match?" | **Part 10c** reconciliation | Confirms the basis alignment before attributing differences to insurance. |

Rule of thumb: **explore in the simple model, optimize in the HJB, confirm in the full model.** Treat a sub-2-SE full-model gap as value-neutral.

---

## 9. The `g_CE` metric — what it is, and why "CE"

`g_CE` is the report's headline decision metric — the single ruin-aware growth rate on which all three models are compared. It appears in every advantage figure (`+2.98 pp/yr`, `+0.79 ± 0.26 pp/yr`, …) and is the quantity the HJB maximizes.

**Definition.**

```
g_CE = exp( (1/T) · mean_paths[ log( max(W_T, $100K) / W_0 ) ] ) − 1
```

Floor each path's terminal equity at the operational ruin threshold ($100K), take the mean of log-growth (= log of the geometric mean), annualize over `T = 25` years, and report a per-year rate. `W_0 = START_EQUITY ≈ $3.89M`.

### What "CE" stands for: Certainty Equivalent

The *certainty equivalent* of a risky payoff is the single **guaranteed** amount you would accept in place of the gamble. Formally, under a transform `u`, the CE of a random terminal wealth `W_T` is `u⁻¹(E[u(W_T)])`. Here the transform is the logarithm, so the certainty-equivalent terminal wealth is the **geometric mean** of the (floored) terminal wealth:

```
W_CE = exp( E[ log max(W_T, $100K) ] )
```

and `g_CE` is just the **constant, deterministic growth rate that turns `W_0` into `W_CE` over `T` years**:

```
g_CE = (W_CE / W_0)^(1/T) − 1.
```

So a strategy with `g_CE = 4%/yr` is one you would be *indifferent to versus a certain 4%/yr* (under the log transform): the entire distribution of risky outcomes — booms, busts, and ruins — is collapsed into one equivalent sure-thing growth rate. That collapse is precisely what lets strategies with very different ruin rates be ranked on a single axis.

### The ergodicity-economics reading (why this is not a utility statement)

The "certainty equivalent" label is inherited from decision theory, where it is tied to a *subjective utility function*. **In this framework it is not.** The logarithm here is the **ergodicity transformation** — forced by the multiplicative dynamics of wealth, not chosen to encode a risk taste — so `E[log W_T]` is the expected value of the *time-average* growth, and `g_CE` is the **ruin-aware time-average (geometric-mean) growth rate**: the deterministic rate one firm actually compounds at along its single path. Read "certainty-equivalent growth" as "**deterministic-equivalent time-average growth**," with no appeal to preferences (consistent with the project's reading of `LogUtility` as the ergodicity transform — see §3 item 9, `hjb_solver.py:252`).

### Why the $100K floor (ruin-awareness)

Under an absorbing ruin barrier the *unconditional* `E[log W_T]` is `−∞` (any path at `log 0`). Flooring at `RUIN_THRESHOLD = $100K` **charges a ruined path the floor** instead of (a) sending the average to `−∞`, or (b) discarding it — which would bias toward the lucky-survivor cohort, the wrong reference class for a decision made *before* you know whether this firm survives. The same `$100K` is the HJB's `E_FLOOR` and every model's ruin trigger, so the floor is *shared, not a free knob*. Rough exchange rate at `W_0 ≈ $3.89M`, `T = 25`: each **1% of ruin probability docks ~0.16 pp/yr** of `g_CE`.

### `g_CE` vs. survival-conditional growth `g`

The notebook also reports survival-conditional growth `g` (geometric mean over *survivors only*, no floor — sometimes tagged "TAG | Survival"). They answer different questions: `g` = "how fast did the firms that *lived* compound?"; `g_CE` = "what all-in, ruin-charged growth should I expect by choosing this strategy *in advance*?" For any single decision (the Part 11 regret curve, every advantage figure) `g_CE` is the correct axis; `g` silently flatters strategies that let their riskiest paths die.

### How it ties the three models together

Because `g_CE` is a pure function of `(terminal_equities, W_0, T)`, it applies *identically* to the simple-cash sim and the full `WidgetManufacturer` — both emit per-path terminal equity. The **HJB** does not emit Monte-Carlo paths, but its growth function is the same object: with terminal `V = log(max(E_T, E_FLOOR))`, `ρ = 0`, and zero running cost, `V(A₀, e₀, 0) = max_policy E[log max(E_T, $100K)]`. Hence the HJB's own optimal-policy certainty equivalent is recoverable straight from the solve:

```
g_CE_HJB(theory) = exp( ( V(A₀, e₀, 0) − log W_0 ) / T ) − 1.
```

The HJB is, by construction, a `g_CE`-**maximizer**; the "HJB g_CE" printed in the comparison tables is obtained by *deploying* its policy `SIR(A, e)` through one of the two simulators (`simulate_hjb_adaptive` or the full modeler) and applying the same helper.

### Implementation

A **notebook-local NumPy helper** (not a library function), defined equivalently in several cells — canonically `_gce(finals, start, years)` (cell 16), a paired-CRN advantage+SE variant `_paired_adv_se` (cell 16), and local copies (`_gce_equity` cell 37, `_gce_one` cells 62/64, `_tavg_gce` cell 63; the locals dodge a name-clash where Part 12 rebinds the global `_gce`). The two spellings `exp(mean(log r)/T) − 1` and `exp(mean(log r))**(1/T) − 1` are algebraically identical. It is *not* part of the installable `ergodic_insurance` package — `ErgodicAnalyzer` computes time-average growth but not this floored certainty-equivalent variant. One wrinkle: the walk-forward cells (47/49) additionally condition on paths being alive at *window start* before flooring the endpoint; the full-horizon definition used everywhere else floors only.

---

## Appendix A — The dynamics, side by side

### A.1 Simple-cash MC, one year (cell 10, `step_year_per_event_scheduled`)

```
revenue           = A · ATR · shock,   shock = exp(σ·z − σ²/2),  σ = 0.30
operating_income  = A · ATR · margin · shock
# per retained event r (occurrence order), gross r_gross = r·(1+LAE):
#   if r_gross > equity_of(A, Lb):  RUIN now  (equity → 0)
# single-loss limited liability (#1598)
#   else:  future_pay[:10] += DEV_FACTORS · (PHI · r_gross)
# after-tax, scheduled (#1597)
A   += PHI · (operating_income − premium) / (1 − k)
# leverage-amplified retained earnings (#1648)
due  = future_pay[0];  A −= due;  roll(future_pay)
# this year's scheduled claim payment
equity = A·(1 − k) − future_pay.sum();   RUIN if equity < $100K
```

### A.2 HJB drift / diffusion / jump (cell 20)

```
# state (A, e), e = equity/A; claim liab Lb = A·max(1 − k − e, 0); base liab k·A permanent
A_dot = PHI·(A·MU − prem(A,SIR))/(1 − k) − β·Lb        # MU = ATR·margin = 0.1875
E_dot = (1 − k)·A_dot + β·Lb
e_dot = (E_dot − e·A_dot) / A
σ_A   = PHI·A·MU·vol/(1 − k);   σ_e = PHI·MU·vol·|1 − k − e|/(1 − k)   # diagonal only
# jump (compound Poisson, rate λ(A)); per event severity X:
L_ret = (1 + LAE)·(min(X, SIR) + max(X − TOWER_TOP, 0))
e    -> e − PHI·L_ret/A         (assets fixed at impact; equity-only, block-diagonal)
ruin atom if  e·A − L_ret < E_FLOOR  ->  value V_RUIN = log(E_FLOOR)
jump_term = λ · E_X[ V(post) − V ]
terminal:  V(A, e, T) = log(max(e·A, E_FLOOR));   ρ = 0,  T = 25
admissible: (1+LAE)·N·SIR ≤ e·A − E_FLOOR,  N ≈ 1.74   ->  SIR ≤ T/N ≈ 0.51·equity
```

### A.3 Full model, one `step()` (abridged; `manufacturer.py:678`)

`insolvency gates → revenue (ASC 606) → working capital (DSO/DIO/DPO) → coordinated limited-liability payment cap (accruals + claims vs cash + facility) → post-payment liquidity check → reserve re-estimation → depreciation → capex → DTL → operating income (net of premiums/losses/recoveries/LAE/development) → collateral`

`(LOC) costs → COGS/OPEX → net income (ASC 740 tax) → close to equity + dividends → growth (scale ATR) → 2-tier solvency (ASC 205-40) → accounting-equation assertion → metrics.`

---

## Appendix B — Key parameters (the $5M calibration)

| Parameter | Symbol / config | Value | Models |
|---|---|---|---|
| Initial assets | `INITIAL_ASSETS` | $5,000,000 | all |
| Asset turnover | `ATR` | 1.5 → revenue $7.5M | all |
| Operating margin | `OPERATING_MARGIN` | 0.125 → EBIT ≈ $937.5K | all |
| Op-income vol | `REL_OP_INC_VOL` | 0.30 | all |
| Tax rate / retention | — | 0.25 / 0.70 | all (full: ASC 740; simple/HJB: flat) |
| After-tax retention | `PHI_RETENTION` | 0.525 | simple, HJB |
| Base-liability ratio | `BASE_LIAB_RATIO` (k) | 0.222 → start e ≈ 0.778 | simple, HJB (full: `initial_base_liability_ratio`) |
| LAE | `LAE_RATIO` | 0.12 | all |
| Claim schedule | `CLAIM_DEV_FACTORS` | `[.10,.20,.20,.15,.10,.08,.07,.05,.03,.02]` (≈3.84 yr) | simple, full |
| HJB paydown | `BETA_PAYDOWN` | ≈ 0.26–0.28/yr (mean-matched) | HJB |
| Severity size exponent | `GAMMA_SEV` | 0.5 | all |
| Wang anchor | `TARGET_LOSS_RATIO` → `WANG_LAMBDA` | 0.65 → **≈ 0.3669** (loadings 1.54/2.84/3.80/4.43×) | all |
| Loss model | attr / large / cat | freq 1.25 / 0.4 / 0.09; LN(10K,5) / LN(450K,2.5) / Pareto(2.1, 800K) | all |
| Tower top | `TOWER_TOP` | $200,000,000 | all |
| Ruin floor | `RUIN_THRESHOLD` / `E_FLOOR` | $100,000 | all |

| Parameter | Symbol / config | Value | Models |
|---|---|---|---|
| Facility (full only) | `working_capital_facility_ratio` | 0.20 of revenue (#1625) | full |
| Collateral (full only) | `sir_collateral_mode` | `letter_of_credit` (#1637) | full |
| HJB grid | `N_A × N_E × N_SIR` | 80 × 100 × 210; `dt = 0.01`25; horizon 25 | HJB |
| N-event bound | `N_EVENTS_BOUND = λ` | ≈ 1.74 → `T/N ≈ 0.51·equity` | HJB |
| Paths | `N_PATHS` | 1,000 (CRN) | simple, full |

*Library `ManufacturerConfig` defaults differ ($10M assets, ATR 0.8, margin 0.08); the values above are the notebook's calibration (cells 5/10/16/49).*

---

## Appendix C — Source map (where to look)

| Topic | Location |
|---|---|
| Simple-cash 2-state step + single-loss liability | cell 10 `simple_step_2state`, `step_year_per_event`, `step_year_per_event_scheduled` |
| Simple-cash CRN engine + thinning + size scaling | cell 10 `simulate_with_crn`, `generate_loss_pool`, `thinning_keep_prob`, `size_severity_factor` |
| Notebook Wang pricer | cell 10 `_wang_g`, `_wang_layer_loss`, `wang_layer_premium`, `layer_loading_multiple`, `WANG_LAMBDA` |
| Tower factory | cell 10 `make_program`, `make_layer_pricers` |
| HJB drift / diffusion / jump setup | cell 20 `company_dynamics`, `company_diffusion`, `make_jump_term_2d`, `sir_multi_loss_feasible` |
| HJB solve + deployment lookup | cell 20 `HJBSolver.solve_finite_horizon`, `hjb_sir_lookup`, `sanitize_hjb_policy_2d` |
| HJB adaptive deployment in simple MC | cell 37 `simulate_hjb_adaptive` |
| Full-model sweep (CRN through `WidgetManufacturer`) | cell 16 `_sweep_full_final_equity` |
| Full model orchestration / `step()` | `manufacturer.py:678`; mixins `manufacturer_{balance_sheet,claims,income,solvency,metrics}.py` |
| HJB solver internals (Hamiltonian, stencils, schemes) | `hjb_solver.py` (`HJBProblem:355`, `solve_finite_horizon:1271`, `LogUtility:252`) |
| Jump operators + retention bounds + aggregate quantile | `hjb_quadrature.py` (`build_equity_jump_operator_2d:256`, `make_jump_term_2d:493`, `single/multi_loss_insolvency_retention_cap:723/770`) |

| Topic | Location |
|---|---|
| Insurance tower allocation | `insurance_program.py:718` `process_claim` |
| Library pricer (NOT used for HJB) | `insurance_pricing.py` (`InsurancePricer`, `LayerPricer:187`) |
| Loss generation | `loss_distributions.py` (`ManufacturingLossGenerator:1025`) |
| Claim development schedule | `claim_development.py:93` `create_long_tail_10yr` |
| Tax / NOL / DTL | `tax_handler.py`; DTL in `manufacturer.py:966` |
| Going-concern insolvency | `manufacturer_solvency.py:361` `check_solvency` |

---

## Appendix D — The jump term, in full

The continuous-time loss operator the solver integrates (per `HamiltonianTerms`, `hjb_solver.py:509`):

```
−∂V/∂t = f(x,u) + μ(x,u)·∇V + ½σ²(x,u)·∇²V + λ·E_X[V(x−L) − V(x)] − ρV
```

In notebook 07: `f = 0`, `ρ = 0`, `μ/σ` from §A.2, and the jump is the only carrier of loss risk. Its construction (`hjb_quadrature.py`):

1. **Atoms.** Each severity component → `(x_atoms, p_atoms)`: lognormal via Gauss-Hermite (32 nodes), Pareto via stratified atoms (300; half equiprobable in the body, half log-spaced in the tail, within-bin value = conditional mean from `LEV` differences).
2. **Per-(SIR, atom) retained loss** (state-independent): `L_gross = (1+LAE)·(min(X,SIR) + (X−TOWER_TOP)+)`; after-tax `L_ret = PHI·L_gross`. Gross drives the ruin test; after-tax drives the surviving-equity shift.
3. **Per assets-slice operator.** For each `A_i`: post-equity `E_i − L_ret`; if `< E_FLOOR` → ruin mass (`p_ruin`), else post-`e = (E_i − L_ret)/A_i` linearly interpolated onto the e-grid. Stack slices block-diagonally (`J_big`), since assets don't move at impact.
4. **Expectation.** `E_X[V(post)] = J_big · V_flat + p_ruin · log(E_FLOOR)`; the matrix-vector product is cached on `V` identity and reused across the control scan. Return `λ·(E_X[V(post)] − V)`.
5. **Size scaling (#1607).** `build_equity_jump_operator_2d_sizescaled` scales each slice's atoms by `(A_i/$5M)^γ` (large/cat) and the large/cat

frequency by `(1−γ)`; at `γ = 0` it reduces bit-for-bit to the flat operator.

**Why each piece is a simplification:** continuous-time Poisson ≠ discrete annual events (a); equity-only block-diagonal ≠ a full balance-sheet shock (b); atoms ≠ continuous severity (c); `min(X,SIR)+(X−tower)+` ≠ the real layered tower with reinstatements/aggregates (d); IMEX/explicit ≠ implicit (e); nearest-snapping ≠ continuous control (f); single-`β` paydown ≠ the 10-year schedule (§5.2). Collectively they make a 25-year, state-dependent stochastic-control problem solvable in minutes — at the cost of the liquidity channel and exact tower/tax detail that Part 10b restores.

---

*Compiled from the committed notebook and library source. If the notebook is re-run with different `GAMMA_SEV`, `TARGET_LOSS_RATIO`, or grid sizes, re-verify Appendix B (the Wang loadings and `T/N` fraction in particular).*